n = 83 K600: Z[τ]-multiplier closure of the factor base, end-to-end cold

2026-10-19

n = 83 K600: Z[τ]-multiplier closure of the factor base, end-to-end cold

Status (2026-10-19): frozen; Tier 1 run RUN_PLACEHOLDER. The protocol and the schedule were committed before any Tier 1 timing. Every number below that is not from a Tier 1 run comes from the cloud-agent VM: it is a **local diagnostic**, not timing evidence.

Question

The n = 83 a = 1 K600 one-target baseline is the rank-probe producer (`koblitz-n83-rankprobe-20261019`, PR #980, run 39214027289): end-to-end cold 166.0 s, `S` 6.390, gap to the tuned rho rT770 64.0 [53.7, 78.0]. Its rank stage is 143.7 s of that, 87%: 600 guided attempts `[a]G − R_j`, one per factor-log column, each costing about 1.69 M root-table probes. The probe law is r / (166 E), where E is the number of root-table entries. It depends only on the stored states, about 83 N^2 for N orbits. It does not depend on how many columns the linear algebra has.

The change keeps about 600 orbits in the root index, so the index, the table density and the probe law stay the same. It groups those orbits into K' = 600 / M linear-algebra columns using the Z[τ]-multiplier closure (crypto `docs/ic/RESEARCH_IC_NOVEL_DIRECTIONS_20261007.md`, A1; measured at n = 23 and n = 41 in crypto `research/ic_leads_20261010/RESULTS_ROUND2.md`, but never in the n = 83 pipeline):

- Orbit (c, α) is the signed Frobenius orbit of α(R_c) = [a]R_c + [b]φ(R_c), with α = a + bτ.
- Its log is (a + bλ)·log R_c mod r.

The rank stage then needs K' relations instead of 600. Does this lower the **end-to-end cold** cost on new targets and new rank seeds, with every relation checked in the group, every column log verified and every scalar verified? This is an iteration under AGENTS.md "Success criterion". A promoted result becomes the new n = 83 row.

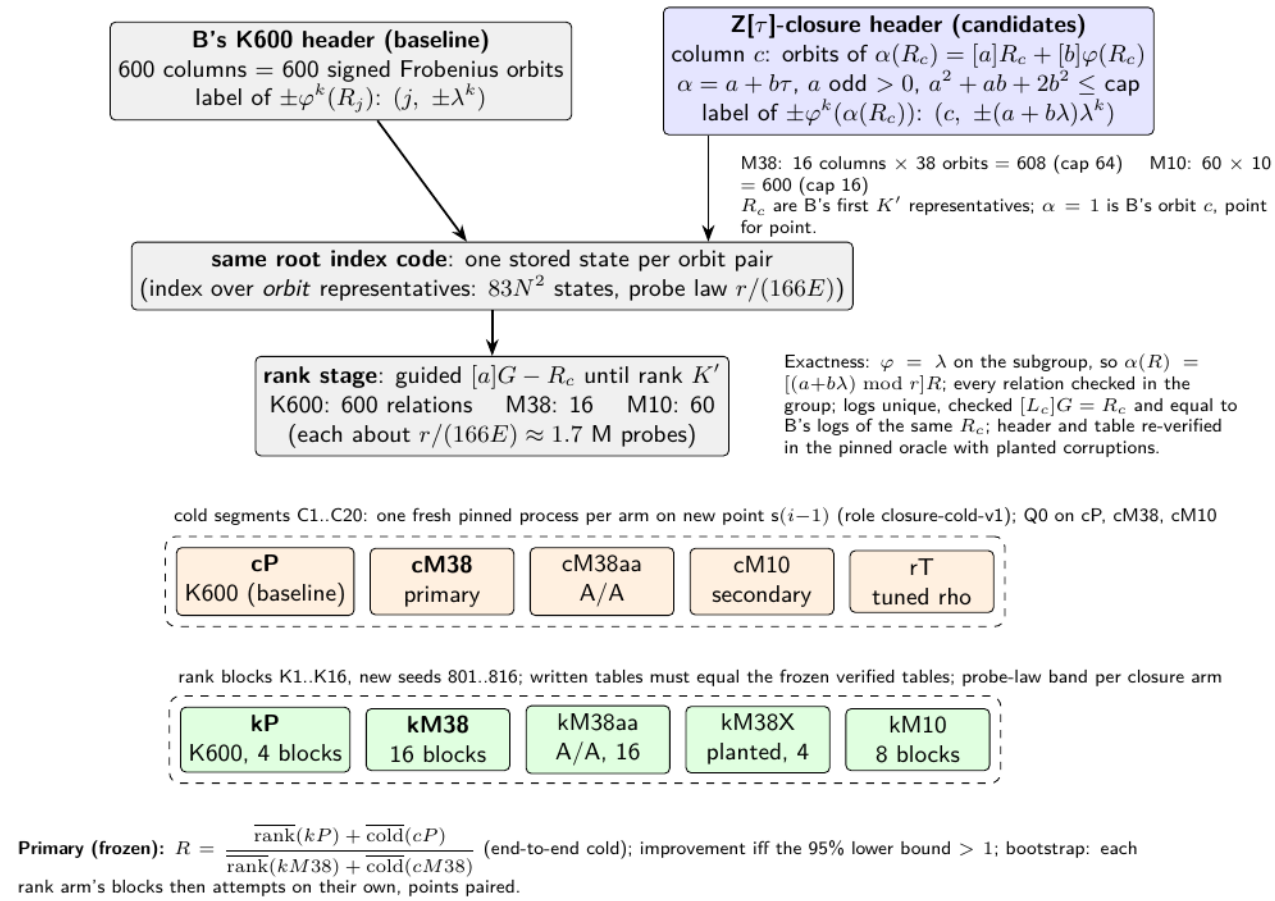


Figure 1: Design (from the frozen protocol; not a result). Source: figures/design.tex, vector: figures/design.pdf.

The closure base

`make_closure_base.py` builds the closure deterministically from B's K600 header (base hash `d4a4da26...`), in the pinned pure-Python oracle:

- **Columns.** R_0 ... R_{K'−1} are B's first K' representatives.
- **Multipliers.** Every $\alpha = a + b\tau$ with a odd, $a > 0$ and $N(\alpha) = a^2 + ab + 2b^2 \le$ cap. On this curve $\tau^2 - \tau + 2 = 0$, and the generator checks $\lambda^2 - \lambda + 2 \equiv 0 \bmod r$.
  - α and $\pm\tau^k \cdot \alpha$ give the same signed Frobenius orbit, and α is divisible by τ exactly when a is even. So "a odd, $a > 0$" picks one α per class.
  - Cap 16 gives M = 10 multipliers; cap 64 gives M = 38.
- **Members and labels.** Orbit (c, α) has the members $\pm\varphi^k(\alpha(R_c))$ for k = 0 ... 82, in B's order. Their labels are $(c, \pm(a + b\lambda)\cdot\lambda^k \bmod r)$.
  - $\alpha = 1$ comes first in each column, so each column's first orbit is B's orbit c, point for point and label for label (checked).
- **Header.** The header keeps B's format and adds:
  - `orbit_representatives`: one per orbit, for the root index;
  - `orbit_labels`: (c, a, b) for each orbit;
  - a `closure` block.

  `factor_base_representatives` holds the K' column representatives. Each header is about 8 MB. It is not committed: `restore.py` regenerates it and checks its frozen SHA-256.

base	columns K'	orbits per column	orbits in the index	points	logical states	stored (folded)	root entries	law r/(166E)
K600 (B, baseline)	600	1	600	99,600	29,880,000	14,940,300	29,878,182	1,727,857
M38 (cap 64, primary)	16	38	608	100,928	30,682,112	15,341,360	30,366,551	1,700,069
M10 (cap 16, secondary)	60	10	600	99,600	29,880,000	14,940,300	29,821,794	1,731,124

The closure bases have slightly fewer distinct root keys per state than K600:

- K600 stores 29,878,182 distinct keys out of 29,880,000 states.
- M10 stores 0.19% fewer keys than K600 on the same number of states.
- M38 stores 1.03% fewer keys than its own state count.

The cause is algebraic coincidences between multipliers inside one column: $(\alpha \pm \tau\hat{}s\ \beta)$ can equal $\pm\tau\hat{}t(\gamma \pm \tau\hat{}u\ \delta)$. The law uses the measured E, so this is priced in. It is about a 1% loss at M = 38.

Independent check of the closure (verify_closure.py, pinned oracle, not the producer and not the generator's code path). It checks:

- identity, and that the column representatives are B's;
- the multiplier set;
- every orbit representative:
  - lies in the prime-order subgroup ([r]P = O);
  - equals [(a + bλ) mod r]R_c, by a scalar multiplication (the generator formed [a]R_c + [b]φ(R_c));
- that all orbits are distinct up to sign and Frobenius (orbit key = the minimum x over the 83 conjugates, across all columns);
- that all |B| x-coordinates are distinct;
- every member and every label;
- the base hash.

With a table, it also checks [L_c]G = R_c for every column, and [(a + bλ)·L_c]G = α(R_c) for every orbit. Three planted corruptions must each be rejected: one label off by one, one duplicated orbit (φ of another) and one log off by one. Both bases pass every check, and every planted corruption is caught (`inputs/local_decisive_test.json`).

The producer change

make_online_source.py derives the producer in two steps:

1. `make_rankprobe_source.py` is the rank-probe experiment's `make_online_source.py`, verbatim, with its five verbatim parents. Its output must hash to the rank-probe protocol's frozen producer digest `d4394df3...`, and the derivation checks this.
2. Four counted, exact replacements:
   - **Index over orbits, logs over columns.** If the header has `orbit_representatives`, the root index is built from them. The columns, the guided rank stage (`[a]G − R_c` for the first pivotless column), the echelon, and the `[L_c]G = R_c` check of every log stay on `factor_base_representatives`. Each orbit representative must be a base point.
   - The labels already carry a coefficient, so `relation_row128` and the descent are unchanged.
   - The summary records `index_orbits` and the header's `closure` block.

- A test that [a]P + [b]φ(P) = [(a + bλ) mod r]P for all 38 norm $\leq$ 64 multipliers, in the producer's arithmetic.

On B's header, which has no `orbit_representatives`, the code path is the rank-probe producer's. The first n = 23 A1 run in crypto built the table from one representative per *column*, and that bug is avoided here by construction: the index gets one representative per orbit.

Local decisive test (cloud-agent VM; diagnostic)

Source: `inputs/local_decisive_test.json`. Setup:

- The producer built from this directory.
- The rank stage with `KIC_RANK_SOLVE=frame` (the baseline's loop), then Q0.
- One full rank block with seed 301 on each base. Then more seeds, to pin the probe law: 302–305 for M10 and 302–309 for M38. Two processes ran at a time on the shared 4-vCPU VM.

	M38 (16 columns)	M10 (60 columns)
seed 301: rank, attempts, relations, failures, rows without gain	16, 16, 16, 0, 0	60, 60, 60, 0, 0
seed 301: mean probes per relation / law	0.719 (s.e. 0.17, 16 attempts)	0.917 (s.e. 0.11, 60 attempts)
all seeds: per-block ratio range	0.577 – 1.347 (9 blocks)	0.849 – 1.176 (5 blocks)
pooled ratio	**0.927 ± 0.082** (144 attempts)	**0.971 ± 0.061** (300 attempts)
every block: rank K', 0 failures, 0 rows without gain	yes	yes
column logs	every [L_c]G = R_c (producer and oracle); equal to B's archived logs of R_0..R_15	the same, R_0..R_59
every orbit's log [(a + bλ)L_c]G = α(R_c) (oracle)	608 / 608	600 / 600
Q0	scalar 3141592653589793 (the archived one), 1,612,533 probes	3141592653589793, 2,130,165 probes
index digests, Q0 identity and log table over all seeds	one each	one each
rank stage, local (ms)	3,785 – 8,299	20,001 – 28,308
ns per probe, local	217 – 241	227 – 234

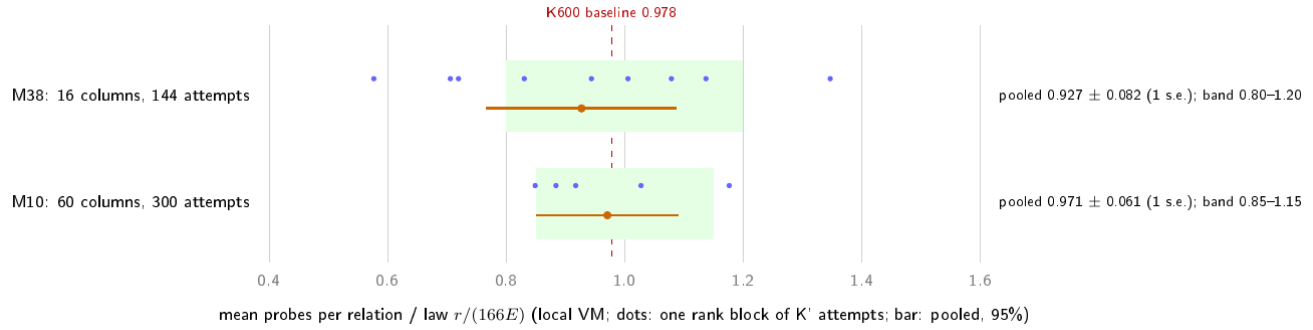


Figure 2: Local probe law (diagnostic): mean probes per relation over law per local rank block (dots) and pooled with a 95% interval, with the frozen gate bands. Source: figures/local_law.tex, rendered by `make_local_figure.py`.

Reading.

- The closure behaves as a set of independent points for the root index. At equal stored states, the probes per relation stay at the law (pooled 0.93 and 0.97, against the baseline's 0.978 on 2,400 runner attempts). The number of relations falls from 600 to 16 or 60.
- Seed 301's M38 block alone (0.72) is outside 0.9–1.1. That is 1.6 standard errors with 16 attempts, and the eight further blocks bring the pooled value to 0.93.
- There is no sign of degeneracy:
  - every orbit is distinct;
  - every closure point is in the subgroup;
  - no relation failed to add rank;
  - every log is unique and equals B's.
- The per-probe cost is the baseline's own loop (227–241 ns locally, against the rank-probe README's 233 ns for the same loop on this VM).

Choice of the primary (pre-registered from this test). M38 is the primary and M10 the pre-registered secondary. Both passed. M38 needs 16 relations instead of 60 at the same law: on the runner that is about 3.8 s of rank stage instead of 14.4 s, so a modelled end-to-end cost of about 26 s against 37 s.

Frozen Tier 1 design

One GitHub `ubuntu-24.04` runner, one pinned CPU, Tier 1 counters, one fresh process per run. Every IC arm runs the closure producer (`ic_online_closure_r770`) with:

- the 16-byte compact table over the folded index;
- lift_scan;
- the Feistel order;
- the prefetch pipeline;
- the frame rank loop.

Arm	Kind	Base	Role
`cP`	cold	K600 + B's table	**baseline** cold (the rank-probe producer's cP)
`kP`	rank	K600	**baseline** rank stage (600 attempts), blocks K1, K5, K9, K13
`cM38`	cold	M38 + its verified table	**primary candidate** cold
`kM38`	rank	M38	**primary candidate** rank stage (16 attempts), all 16 blocks
`cM38aa`, `kM38aa`	cold, rank	M38	A/A of the primary
`kM38X`	rank	M38	planted positive control (`frame_plant`, a known ~2x per-attempt cost), blocks K1, K5, K9, K13
`cM10`, `kM10`	cold, rank	M10	pre-registered secondary (60 attempts), the 8 odd blocks; never decides
`rT`	rho	—	tuned rho rT770 (Fold/64/DP6), seed 910000 + point seed

- **Cold panel.** 20 new public points (`inputs/cold_targets.json`, role `closure-cold-v1`, built by `make_targets.py`). They are disjoint from every earlier n = 83 panel of the law, all regenerated:
  - Q0, B's panel and the scan-order, combined, probe-pipeline, state-fold, table-filter, stacked, rank-cut and rank-probe panels;
  - the 64 n131-ref-walk `eval` points;
  - the panels registered by the concurrent lanes:
    - `indexlift-cold-v1` (claude/n83-indexlift);

  - * `k899-cold-v1` (claude/n83-k899);
  - * `rotkey-cold-v1` (claude/n83-rotkey);
  - * `onlineframe-v1` (claude/n83-onlineframe, 64 points).

  Each was checked equal to the file committed in its worktree.

  No scalar is known for any of the points.
- **Rank blocks.** K1–K16 use the new rank seeds 801–816. Seeds already used are 101–102, 201–204, 301–304, 401–404, 501–504 and 701, and the local test used 301–309. Each block ends with one target, its unit s000–s015.
- **Segment order.** O1 (Q0 on cP, cM38 and cM10), then C1 K1 C2 K2 ... C16 K16 C17 ... C20. The order within each segment is shuffled with new seeds. The time guard is 300 minutes.

Primary (pre-registered): end-to-end cold.

- E2E(k, c) = the mean rank stage of rank arm k over its blocks, plus the mean cold time (setup + one-target online, cached table) of cold arm c over the 20 points.
- R = E2E(kP, cP) / E2E(kM38, cM38).
- Bootstrap, 10,000 draws, seed 20261019. Each rank arm's blocks are resampled, then its attempts within each drawn block, independently for the two arms (the baseline's and the candidate's attempts are different targets). The stage's non-attempt time is kept. The points are resampled, paired across cP and cM38.
- **Improvement** iff the 95% lower bound of R exceeds 1; regression iff the upper bound is below 1; otherwise not separated. **Invalid** if any gate or control fails or any scheduled row is missing.

Controls.

- A/A: E2E(kM38, cM38) / E2E(kM38aa, cM38aa), with blocks and attempts paired, and cold cM38 / cM38aa. Both must be inside [0.97, 1.03].
- Planted timing control: the per-attempt rank cost kM38X / kM38 must have its lower bound above 1.
- Planted exactness controls: the three corruptions must each be rejected by `verify_closure.py` for both closure bases, before timing.

Gates. B's index-digest and "table equal to B's" gates cannot apply to a different base. They are replaced by the following, all frozen before timing:

1. **Bases and tables verified independently before any timing** (`check_log_table.py`):
   - B's table, in the pinned oracle (as before);
   - each closure header, regenerated to its frozen SHA-256 and passed by `verify_closure.py`;
   - each closure table, with [L_c]G = R_c for every column and [w·L_c]G = α(R_c) for every orbit;
   - the planted corruptions, rejected;
   - the closure logs, equal to B's archived logs of the same R_c.

   Logs are unique, so this last check ties two independent rank stages together.
2. **Every IC process** matches its base's frozen record: base hash, columns, orbits in the index, logical and stored states, slots, root entries, and the index digests. These are B's for K600; for M38 and M10 they are the values the local test reproduced over every seed.
3. **Per point:** one scalar across every IC process on all three bases and rT, replayed as [d]G = Q in the pinned oracle. One relation and one probe count per point and base.
4. **Q0:** cP gives B's archived Feistel identity (1,987,852 probes). cM38 and cM10 give the archived scalar and their frozen Q0 relation and probe count.
5. **Rank arms:**
   - full rank K', with 0 failed attempts;
   - one frame per attempt;
   - the producer's own [L_c]G = R_c check;
   - the written table equal to the base's frozen verified table, log for log.
6. **Per block:** one relation digest, one per-attempt probe vector, one root-call count and one set of point-addition counters across kM38, kM38aa and kM38X.
7. **Probe law:** the pooled mean probes per relation of kM38 (256 attempts) and of kM10 (480), over law r / (166 E), must lie inside [0.80, 1.20] and [0.85, 1.15] respectively. That is about 3.3 standard errors at the baseline's attempt CV of 0.97. Simulated pass rate under the true law: 0.9985. Rejection rate for a law 1.5x worse: 0.9985 and 1.0.

Why the result stays exact.

- φ acts on the prime-order subgroup as λ, so $\alpha(R) = [(a + b\lambda) \bmod r]R$. Every member's label is therefore its log relative to its column. `verify_closure.py` re-derives every member and label in the oracle and checks each orbit against [w·L_c]G directly.
- A relation is found by the unchanged S3 search and accepted only by the unchanged `lift`: four base points must sum, in the group, to the attempt's point. So each row is an exact identity in the logs.
- Full rank gives the unique logs. Each is checked in process, against the oracle-verified table, and against B's archived logs of the same representatives.
- Every target scalar is checked as [d]G = Q in process and replayed in the oracle. It must equal the baseline's scalar and the tuned rho's.
- Nothing in the search depends on the column grouping. It only changes which unknowns the rows hold.

Power (`power_check.py`, `analysis/power.json`). The prior is `inputs/power_prior.json`, built by `make_power_prior.py` from run 39214027289's kP / kPaa attempts and cP / cR cold A/A pairs. The candidate's attempts are drawn from the baseline's attempts (the same law and loop), scaled by a cost factor that sets the end-to-end effect E. There were 200 simulations per cell, with 300 bootstrap draws each.

effect E (end to end)	M38 (16 blocks)	M10 (8 blocks)
1.00 (no gain; type I)	0.02	0.00
1.10	0.23	0.435
1.50 (minimal effect of interest)	**1.00**	**1.00**
2.00	1.00	1.00
modelled (closure keeps the law): median R	1.00 (R about 6.33)	1.00 (R about 4.52)

The minimal effect of interest is 1.5. The mechanism divides the relations by M, so an end-to-end effect below 1.5 would mean the closure lost most of its mechanism. The law gate reports that on its own.

Runtime estimate.

- Build and tests: about 15 minutes.
- Closure headers and their verification: about 3 minutes.
- Cold segment: about 95 s.
- Rank block: about 250 s with kP, about 70 s without.
- In total about 1.3 hours, and up to about 2 hours on a slower runner.

Not included.

- The optional x86-64-v3 / `+pclmulqdq` build arm (survey lead L6) was not added: one binary for every arm keeps the per-probe cost identical across baseline and candidates.
- Factor-base construction is charged to no arm, the convention of both n = 83 rows. A closure header holds B's first K' orbits plus (M − 1)·K' images, each costing two small scalar multiplications.

Result

Pending (run RUN_PLACEHOLDER).

Files

File	Role
`protocol.json`, `make_protocol.py`, `schedule.json`, `make_schedule.py`	frozen protocol, its builder, frozen run order
`make_online_source.py`, `make_rankprobe_source.py`, `make_rankcut_source.py`, `make_stacked_source.py`, `make_tablefilter_source.py`, `make_probepipe_source.py`, `make_combined_source.py`, `online/`, `lift2/`	the closure producer (the rank-probe derivation verbatim, digest-checked, plus the closure header path); tuned rho; the two-limb canonicalizer crate

File	Role
make_closure_base.py, verify_closure.py	closure header generator; independent verifier of the header and its factor-log table, with planted corruptions
restore.py, check_log_table.py	restore and digest-check crypto 770f6a69, the closure headers and every local source; verify B's table and both closures before timing
inputs/factor_logs_n83_closure_M38.json, inputs/factor_logs_n83_closure_M10.json	the closure factor-log tables (written by the local seed-301 rank stage; verified in the oracle)
inputs/local_decisive_test.json	the local decisive test (frozen input: the closure bases' index digests, root entries and Q0 identities)
make_targets.py, inputs/cold_targets.json	the new cold panel
make_power_prior.py, inputs/power_prior.json, power_check.py, analysis/power.json	power prior (run 39214027289), simulation, result
arm.py, run_schedule.py	one arm per pinned process; the schedule runner
verify_receipt.py	gates, statistics, decision rule; result.json, result_table.md
figures/design.*, figures/local_law.*, make_local_figure.py	design diagram; local probe-law graph (TikZ source, PDF, PNG)
make_figures.py	result figure, after the run
smoke/	local diagnostics: runner smoke, tests, verifier statistics on synthetic rows
README.pdf	this report: pandocREADME.md-oREADME.pdf--pdf-engine=pdflatex-Hreport_header.tex--lua-filterreport_filter.lua-Vgeometry:margin=2cm-Vfontsize=10pt-Vcolorlinks=true